TriVerity

Evidence of AI tool usage

Team OeroAI · Tencent Cloud AI Hackathon Singapore 2026 · Digital Native Track (Ryde challenge)
Project: TriVerity: Multi-Agent Autonomous Dispute Resolution System · https://github.com/yuzhihao1212/TriVerity

This document collects screenshots of the Tencent tools we used while building TriVerity, each with a short description of what we asked and what the tool produced.

Tool	What we used it for	Screenshots
Tencent Cloud CodeBuddy	AI coding assistant in the IDE	7
Tencent WorkBuddy	AI agent workspace (model: Hy4 preview)	3
Miora	AI image design (model: Hy Image 3.5 preview)	4 + final cover

We also used the AI coding assistants Claude Code, OpenAI Codex and Kiro for features and code review; every change went through a pull request, automated tests and CI.

Tencent Cloud CodeBuddy

AI coding assistant in the IDE

We used CodeBuddy in agent mode to build the core of TriVerity's AI layer: the shared LLM client for Tencent Hy3, the rider and driver advocate agents, the Hy3 model choice through TokenHub, and the Fraud & Bad-Faith detector with its rules, UI and docs. Each task ended with the test suite run and passing. The prompts below are quoted from CodeBuddy's own conversation export (“Add shared LLM client with offline fallback”, Craft mode, 25–26 Sep); all 12 prompts are listed in the appendix.

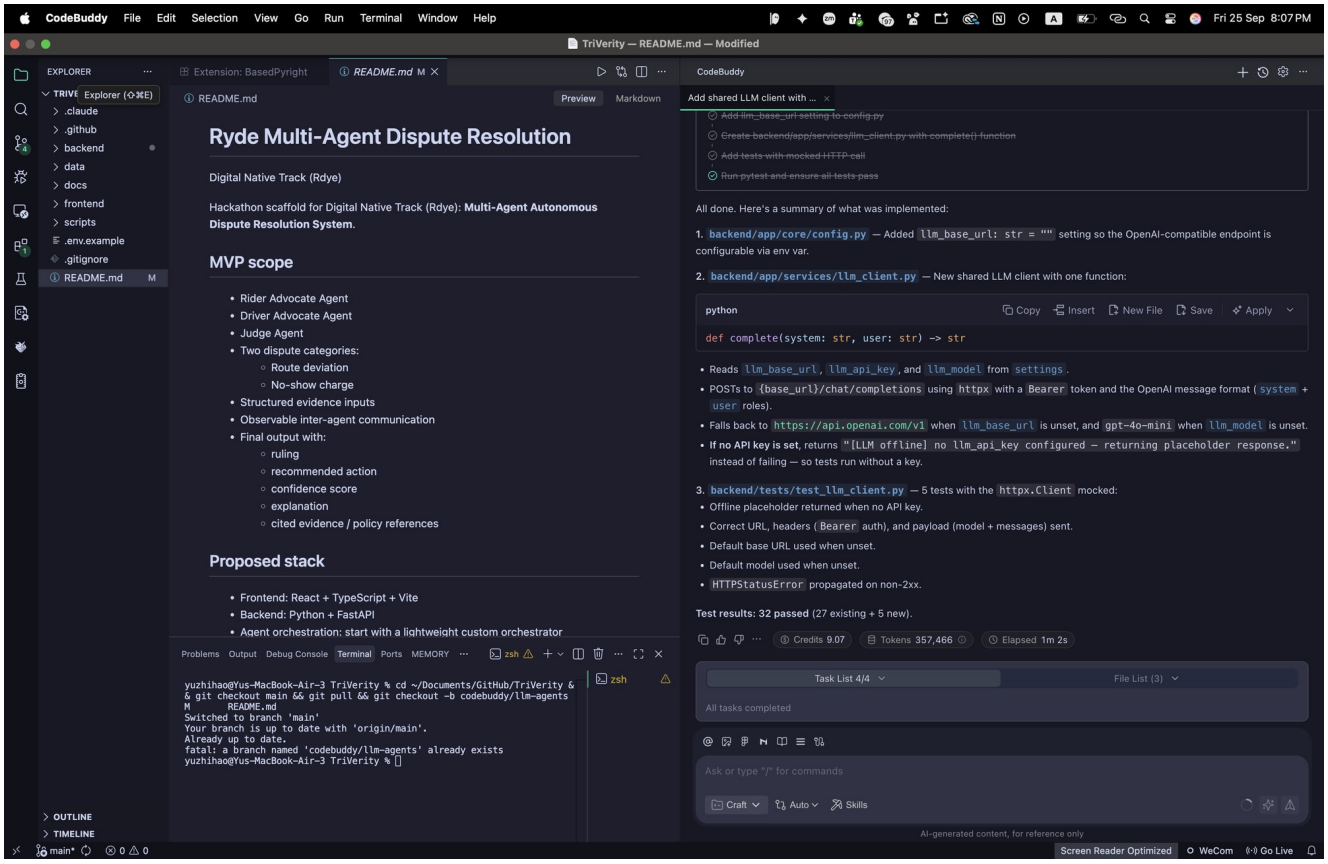


Figure 1. Shared LLM client for Tencent Hy3 (25 Sep, prompt 1). Prompt: “Read README.md, docs/architecture.md and backend/app/ to understand the project. Then add backend/app/services/llm_client.py: one function complete(system, user) that the agents will share… If no API key is set, return a clearly-marked offline placeholder instead of failing, so tests run without a key.” CodeBuddy added the llm_base_url setting, the shared client and 5 unit tests; the backend suite passed (32 tests).

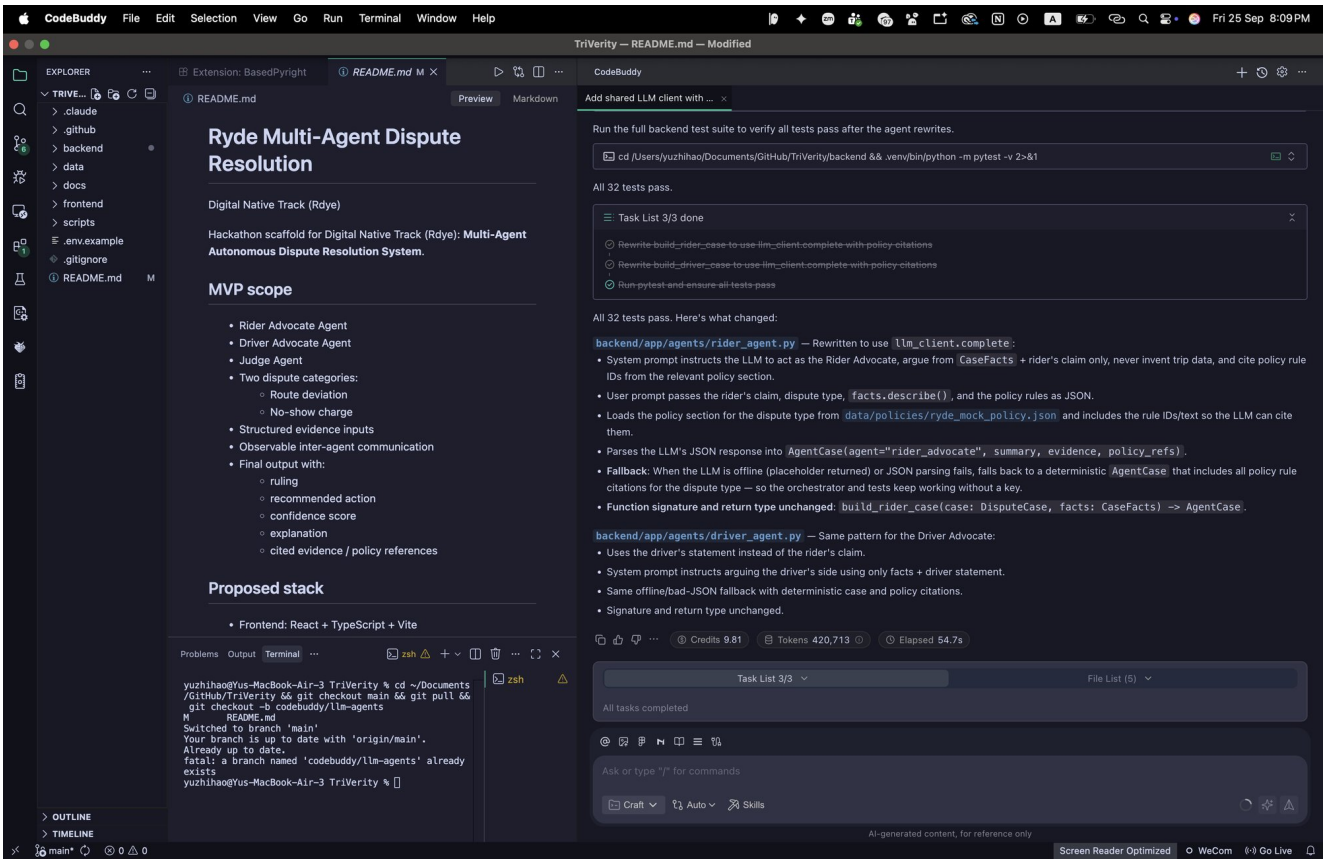


Figure 2. Rider and driver advocate agents (25 Sep, prompt 2). Prompt: “Rewrite build_rider_case… to use llm_client.complete. The rider advocate argues the rider's side using only CaseFacts and the rider's claim, never inventing trip data, and cites rule IDs from data/policies/ryde_mock_policy.json… Then do the same for driver_agent.py. Keep pytest passing.” CodeBuddy rewrote both advocates and ran the suite (32 passed).

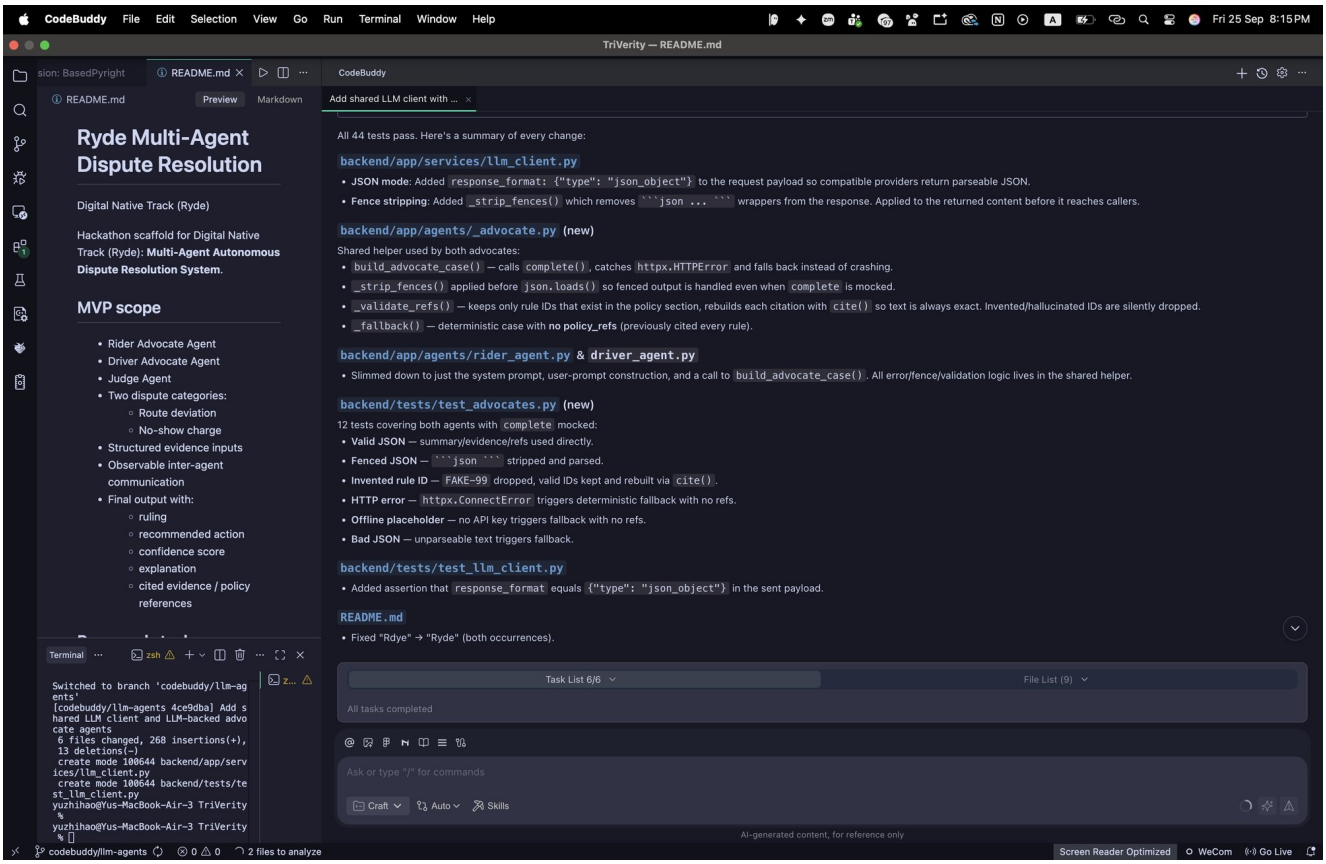


Figure 3. Reviewing and hardening the advocates (25 Sep, prompt 3). Prompt: “Review and fix the LLM advocate agents… If complete() raises an httpx.HTTPError, fall back to the deterministic case… request JSON mode… and strip code fences… Validate policy_refs: keep only rule IDs that exist.” CodeBuddy added JSON mode, a shared _advocate.py helper and 12 advocate tests (44 passing), then committed.

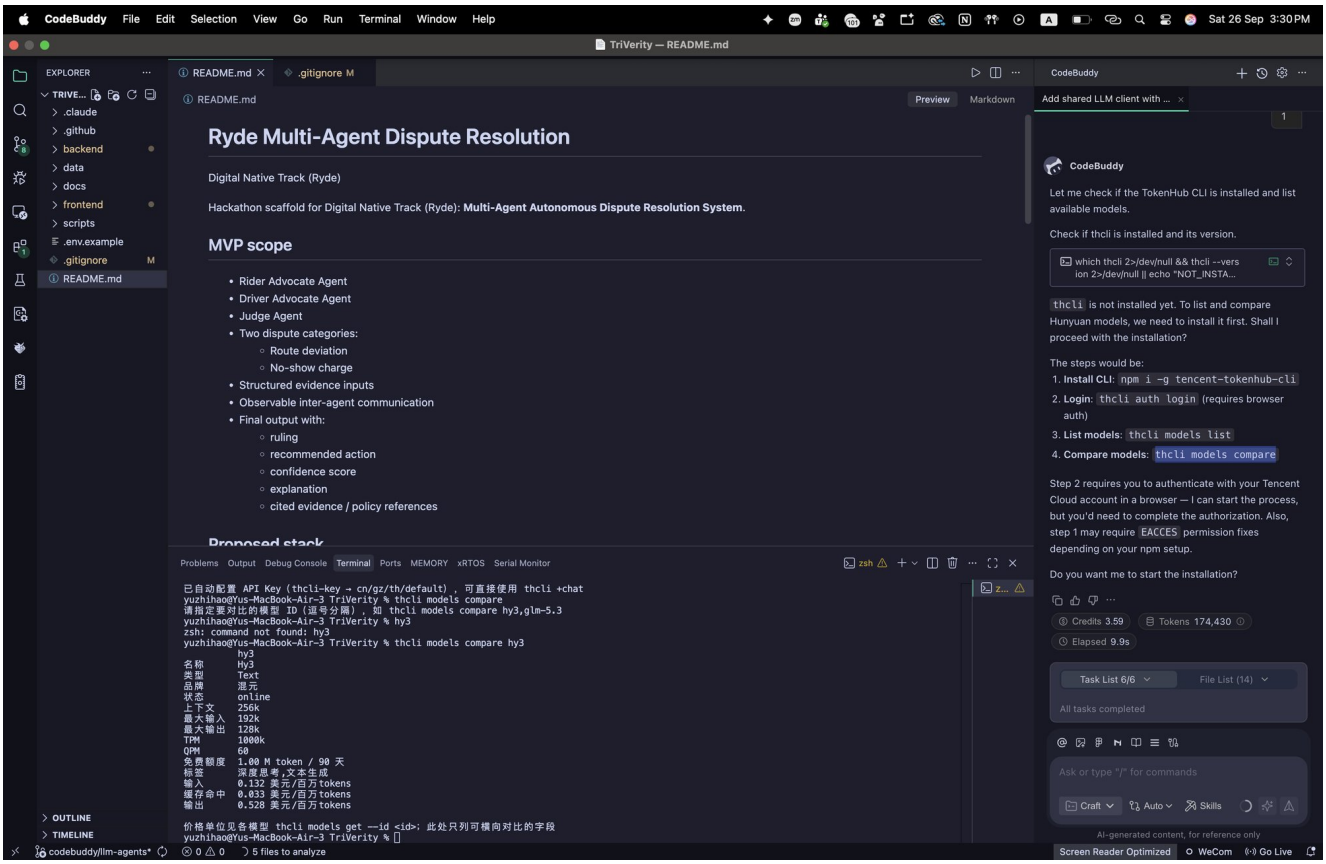


Figure 4. Choosing the model: Hy3 via Tencent Cloud TokenHub (prompts 6–8). Prompts: “Please read the following document to help me install TokenHub CLI”, then “Are you able to check which model from Hy is best for this application?” CodeBuddy walked through installing the TokenHub CLI and compared models; the terminal shows Hy3's limits and pricing, which informed our choice of Hy3.

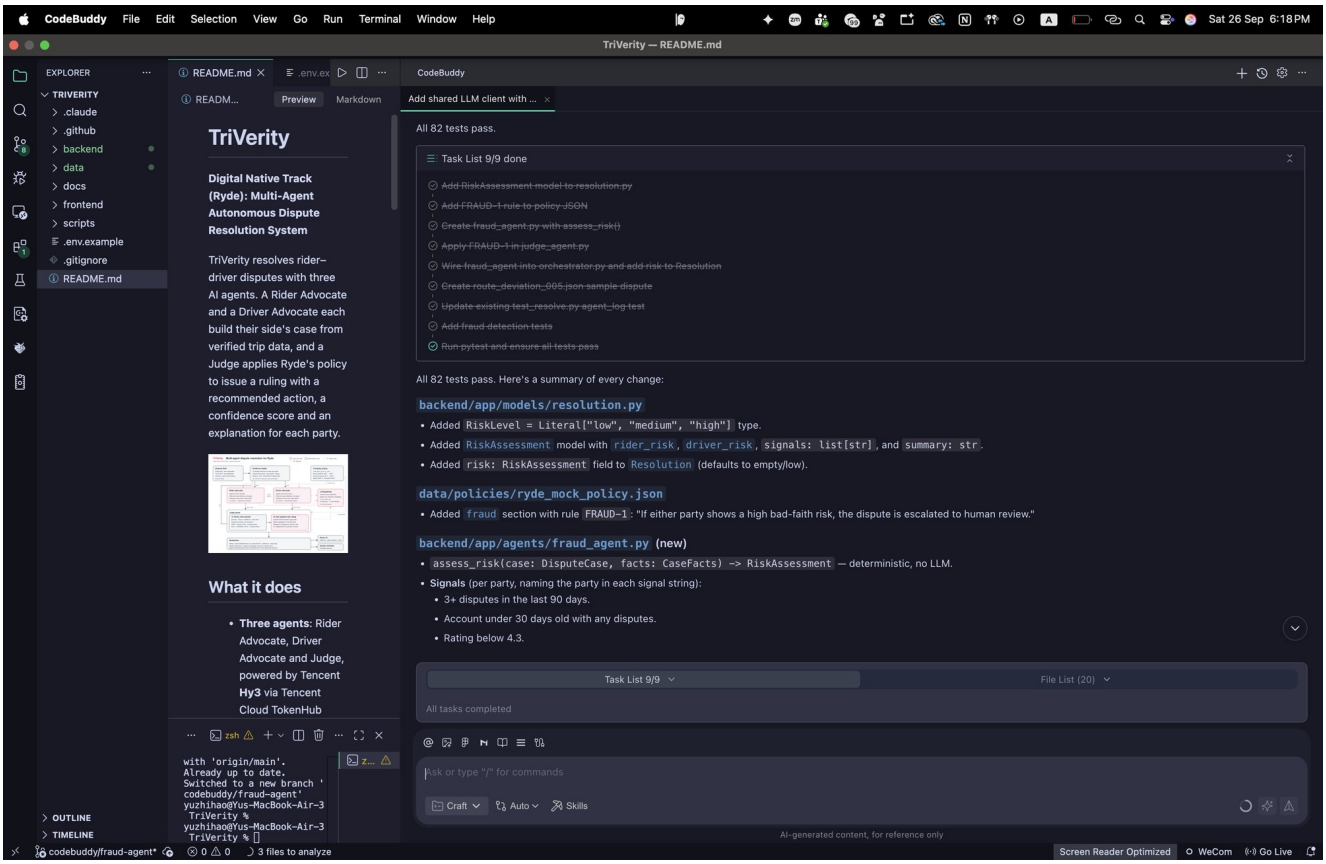


Figure 5. Fraud & Bad-Faith Detection agent (26 Sep, prompt 9). Prompt: “Add a Fraud & Bad-Faith Detection agent to the TriVerity backend… add assess_risk(case, facts) -> RiskAssessment… with rider_risk and driver_risk (low, medium, high), signals and summary. Scoring must be deterministic.” CodeBuddy added the agent, the model and the FRAUD-1 policy rule; 82 tests passing.

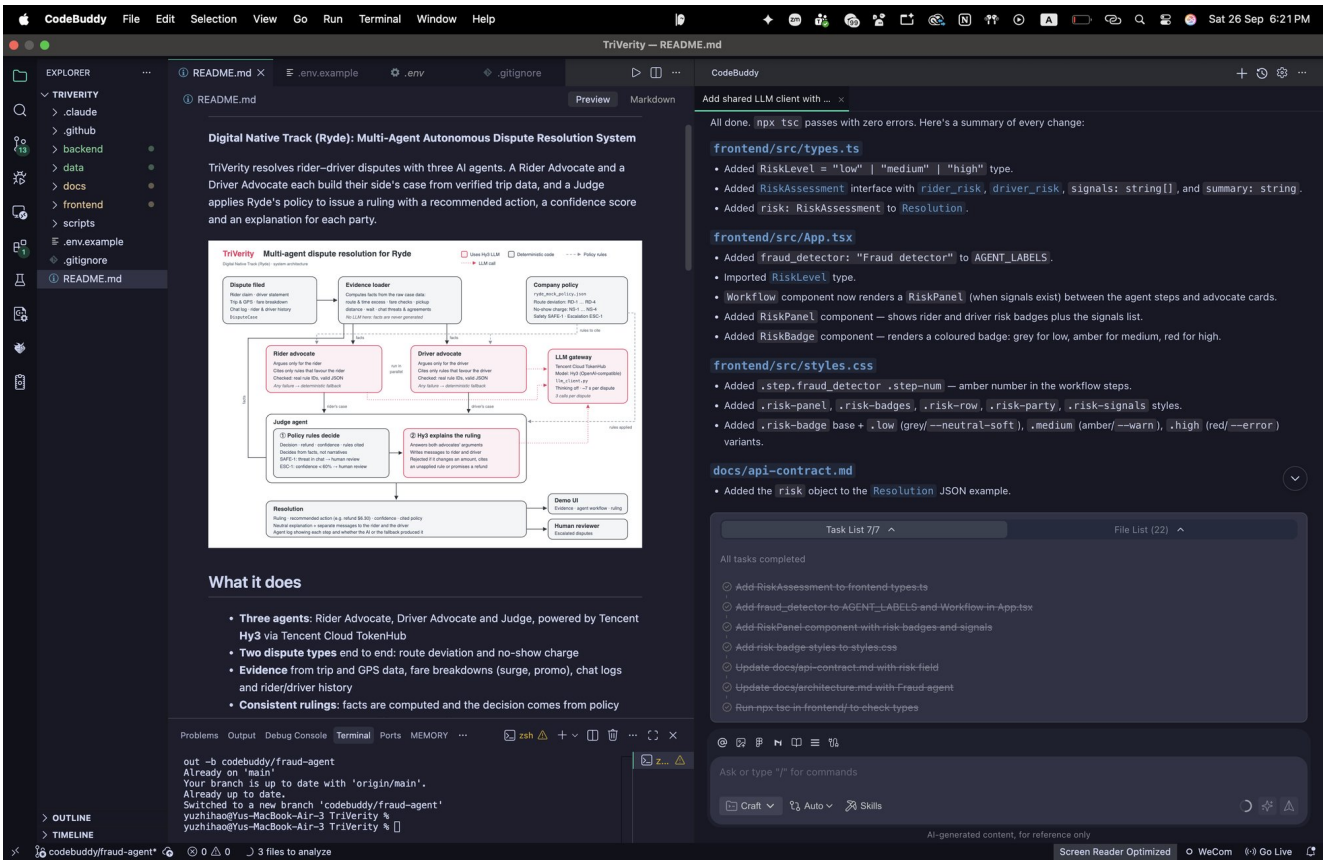


Figure 6. Fraud assessment in the UI and docs (26 Sep, prompt 10). Prompt: “Show the fraud assessment in frontend/src/App.tsx: a ‘Fraud detector’ step in the agent workflow and a small risk panel with… coloured badges (low grey, medium amber, high red)… Run npx tsc.” CodeBuddy added the types, risk panel, badges and styles, and updated the API contract and architecture docs; type checks passed.

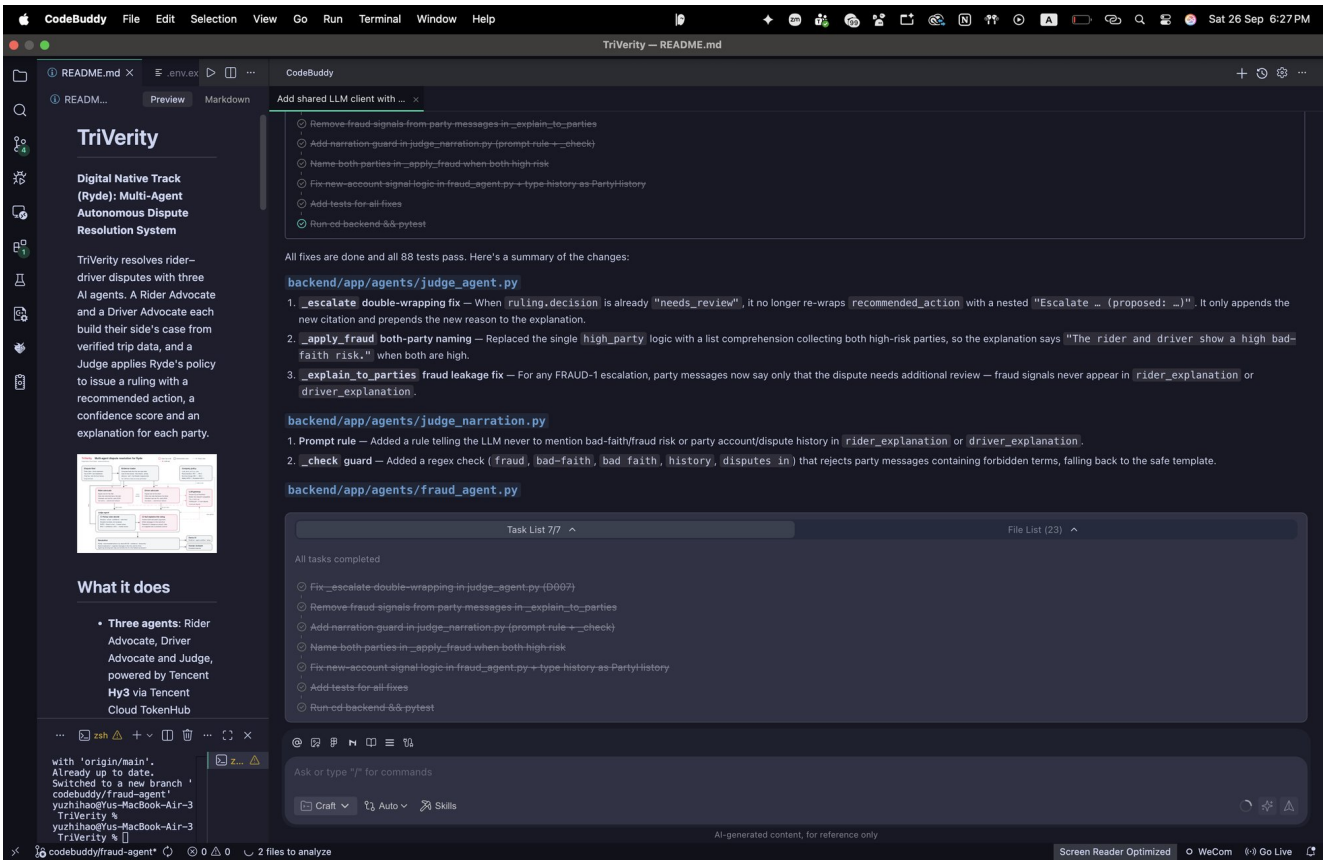


Figure 7. Judge and fraud fixes (26 Sep, prompt 12). Prompt: “Fix these issues in the fraud agent branch: _escalate wraps an already-escalated action twice… Fraud signals must never appear in the rider or driver messages, only in the internal explanation.” CodeBuddy fixed the escalation and added a prompt rule plus a code check for party messages; all 88 tests passing.

Tencent WorkBuddy

AI agent workspace (model: Hy4 preview)

We used WorkBuddy as a read-only reviewer of our public repository: it polished the submission text, checked every doc for inconsistent facts, prepared judge Q&A; and summarised our build history. We verified its output against the code before using it.

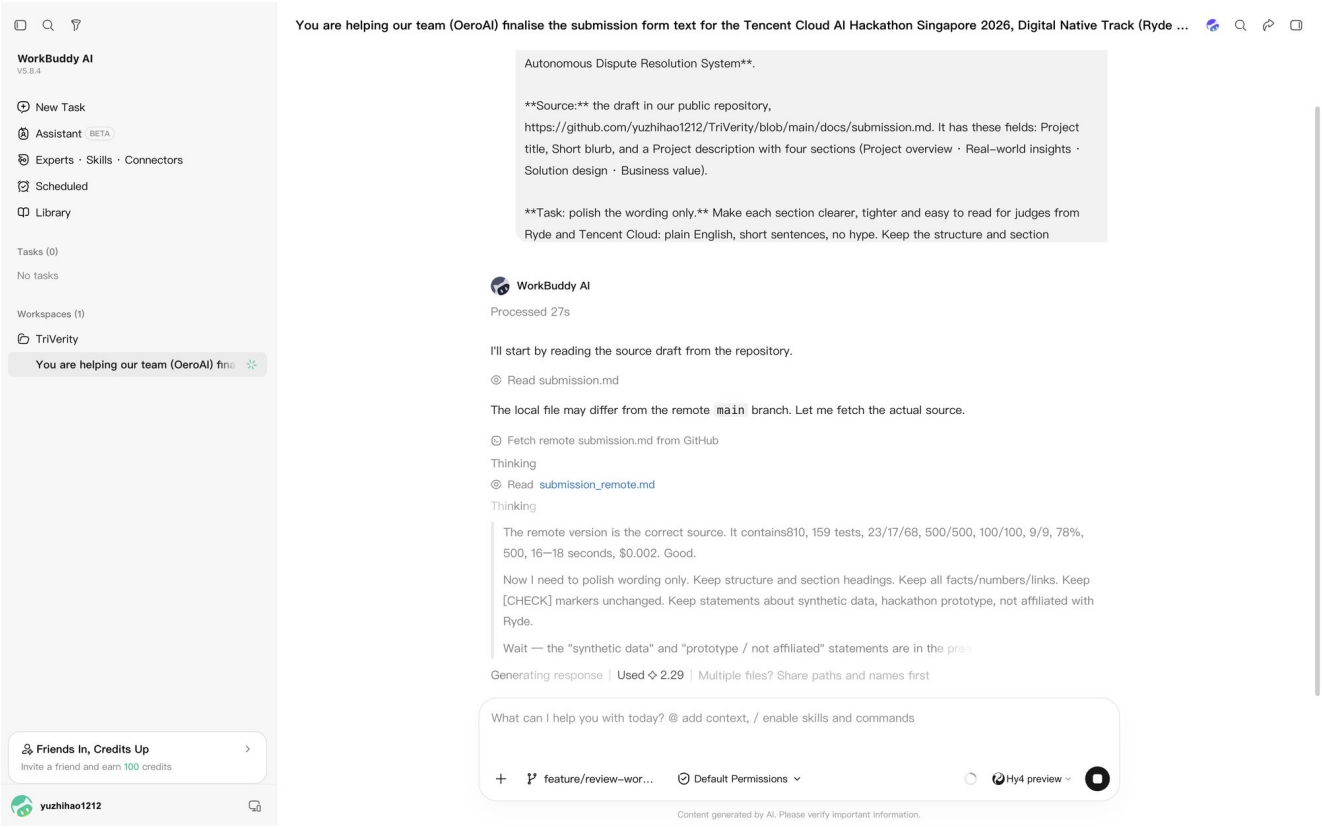


Figure 8. Reading the repository to polish the submission text. Prompt: polish our submission text without changing any fact. WorkBuddy read docs/submission.md, fetched the latest version from GitHub and compared them, using the Hy4 preview model.

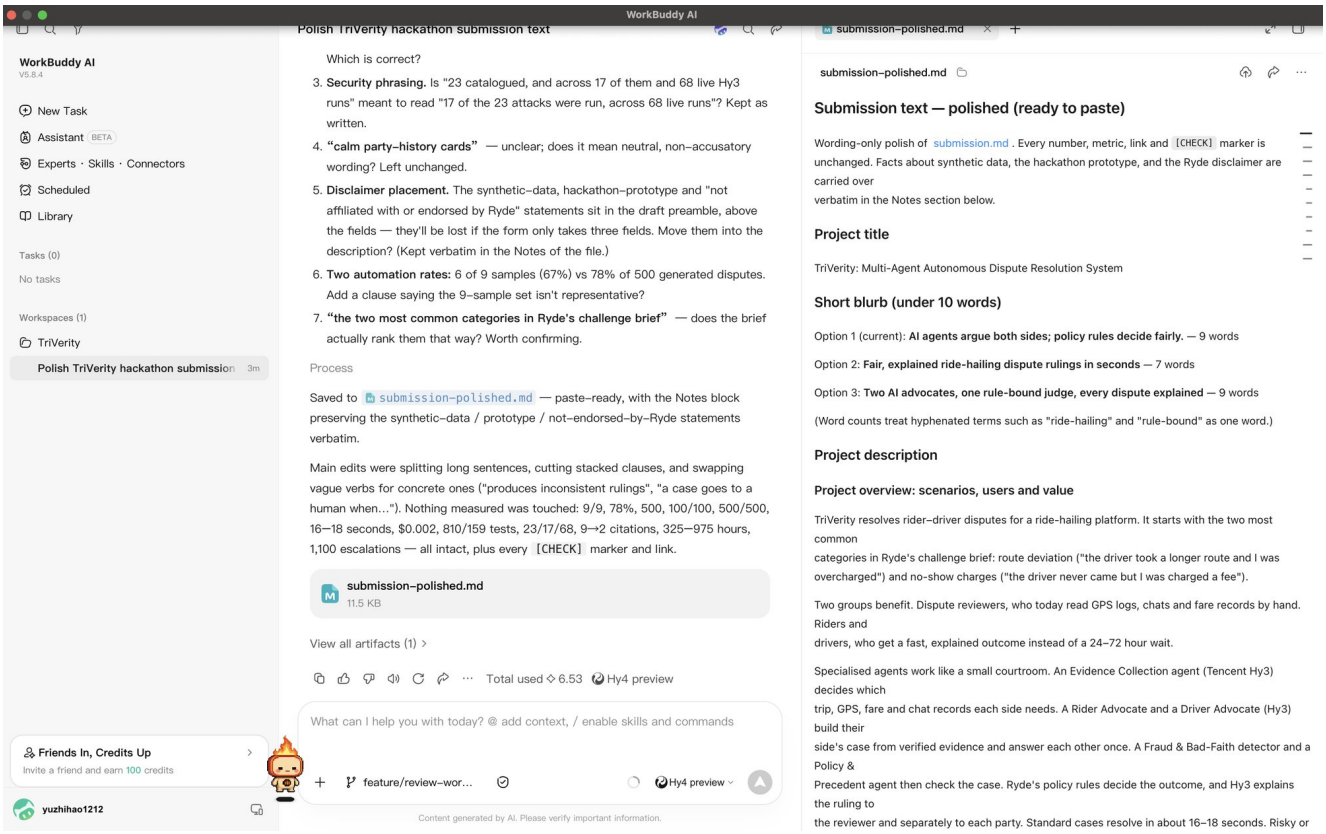


Figure 9. Polished submission text and clarifying questions. WorkBuddy saved a polished draft (title, three blurbs under 10 words, description) and listed seven questions; five of them exposed real issues in our draft, which we fixed.

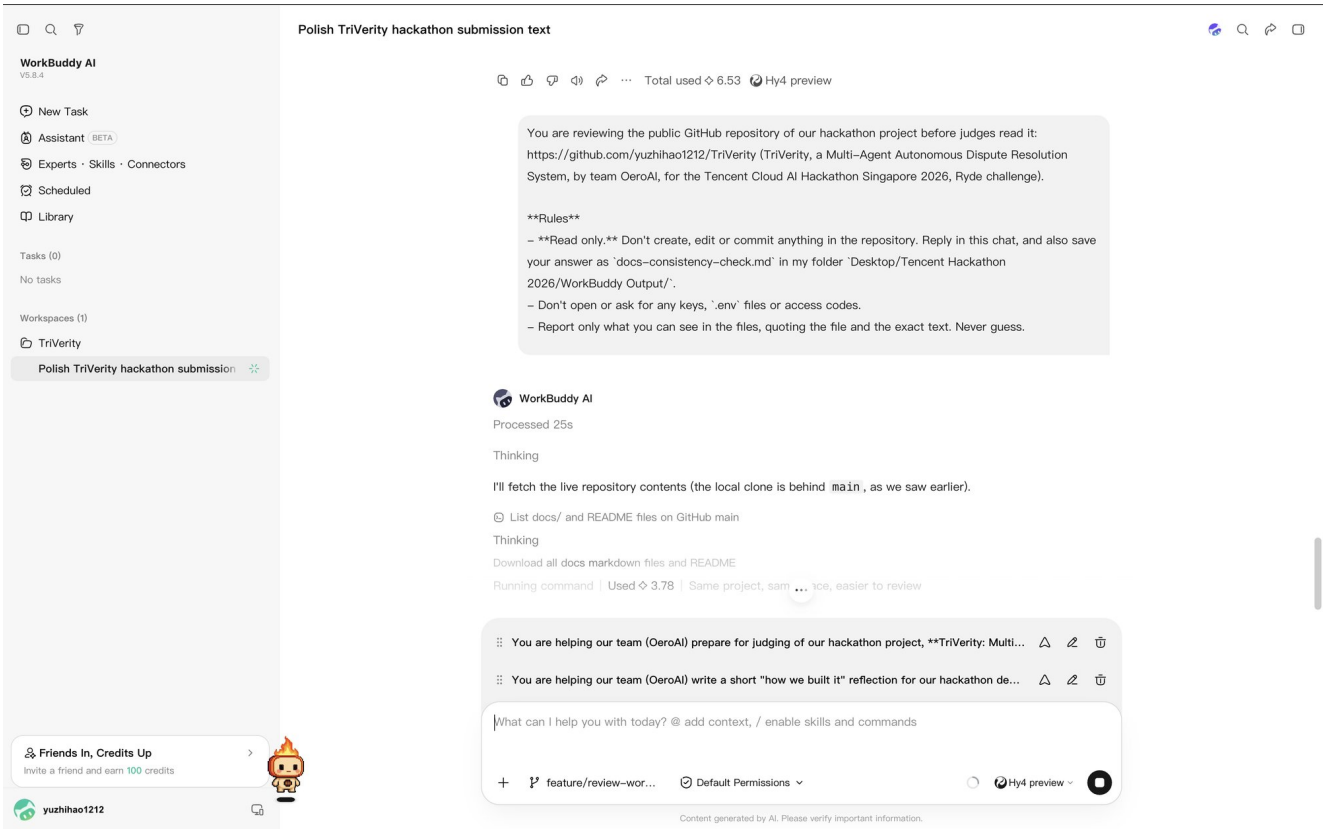


Figure 10. Docs consistency review, with judge Q&A; and timeline queued. A read-only review of the README and all docs for conflicting facts (no keys or .env access), with two more prompts queued: judge Q&A; preparation and a build-history timeline.

Miora

AI image design (model: Hy Image 3.5 preview)

We used Miora to design TriVerity's cover image through an initial design and three guided revisions, ending with the 380 × 216 cover used in the submission.

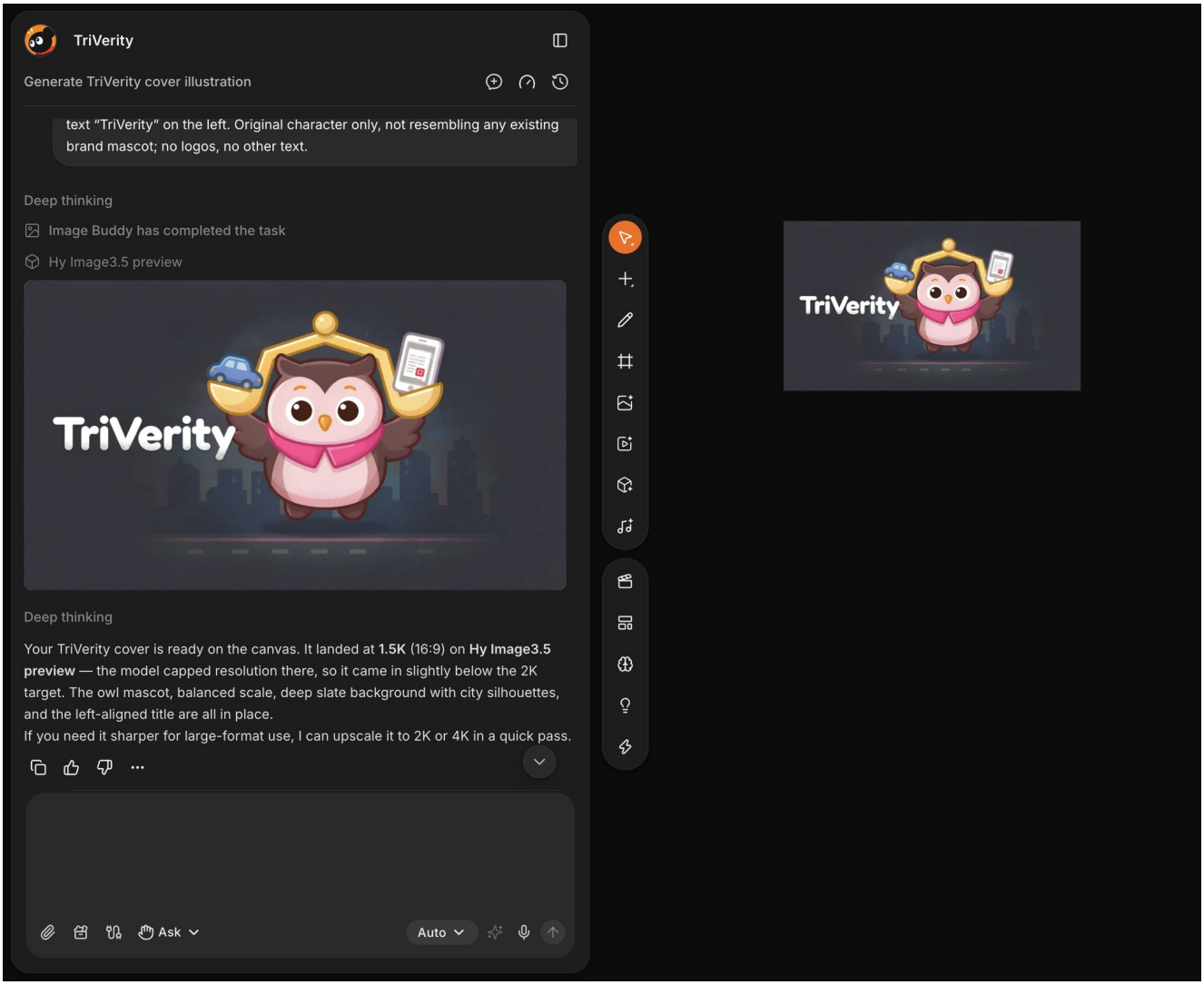


Figure 11. Initial design. An original owl mascot (no existing brand or logo) on a balance scale with a car and a phone, and the “TriVerity” title, in 16:9.

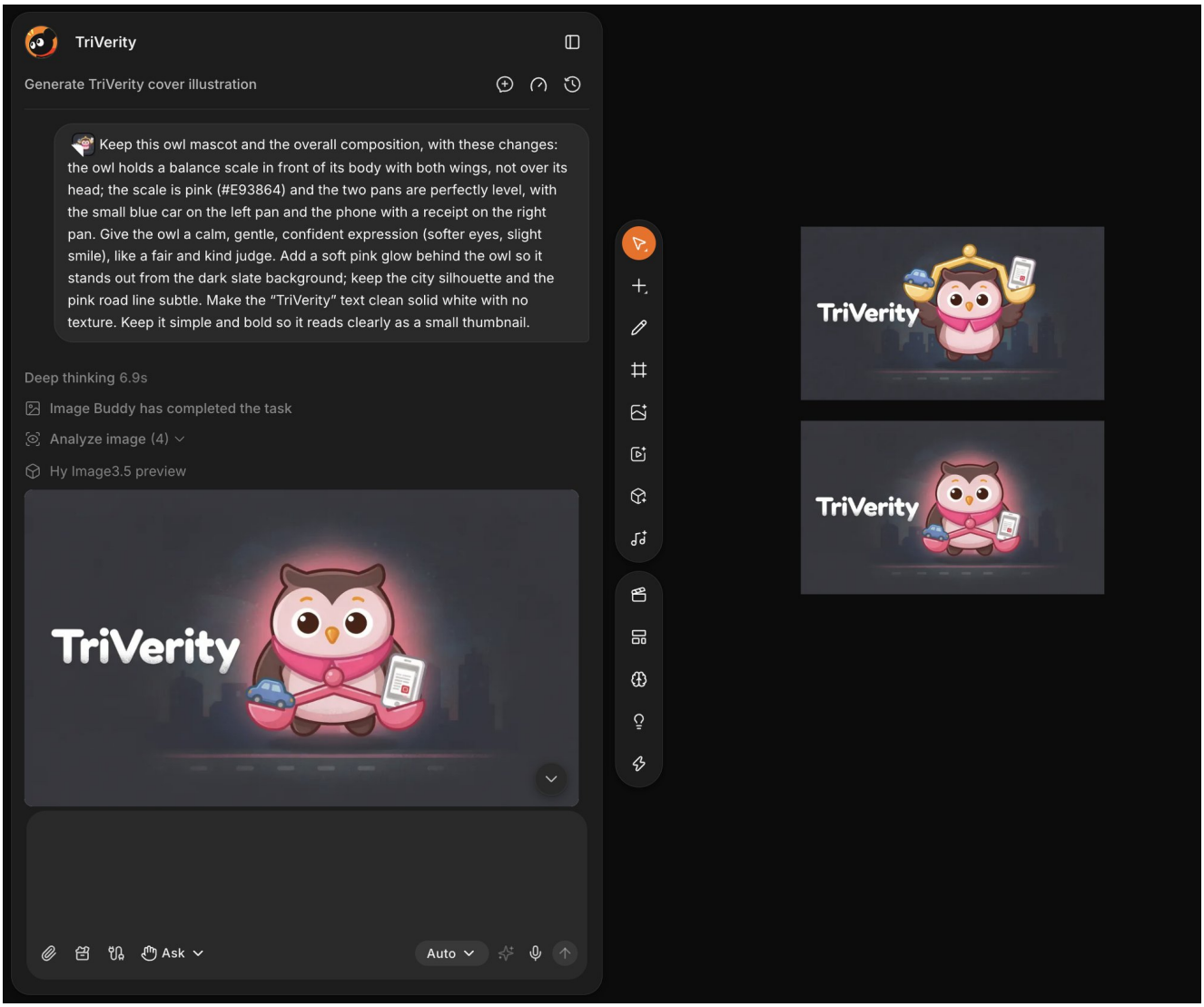


Figure 12. First revision. We asked Miora to keep the mascot and composition, move the scale in front of the owl in our accent pink (#E93864), give it a calm, confident expression and a clean white title.

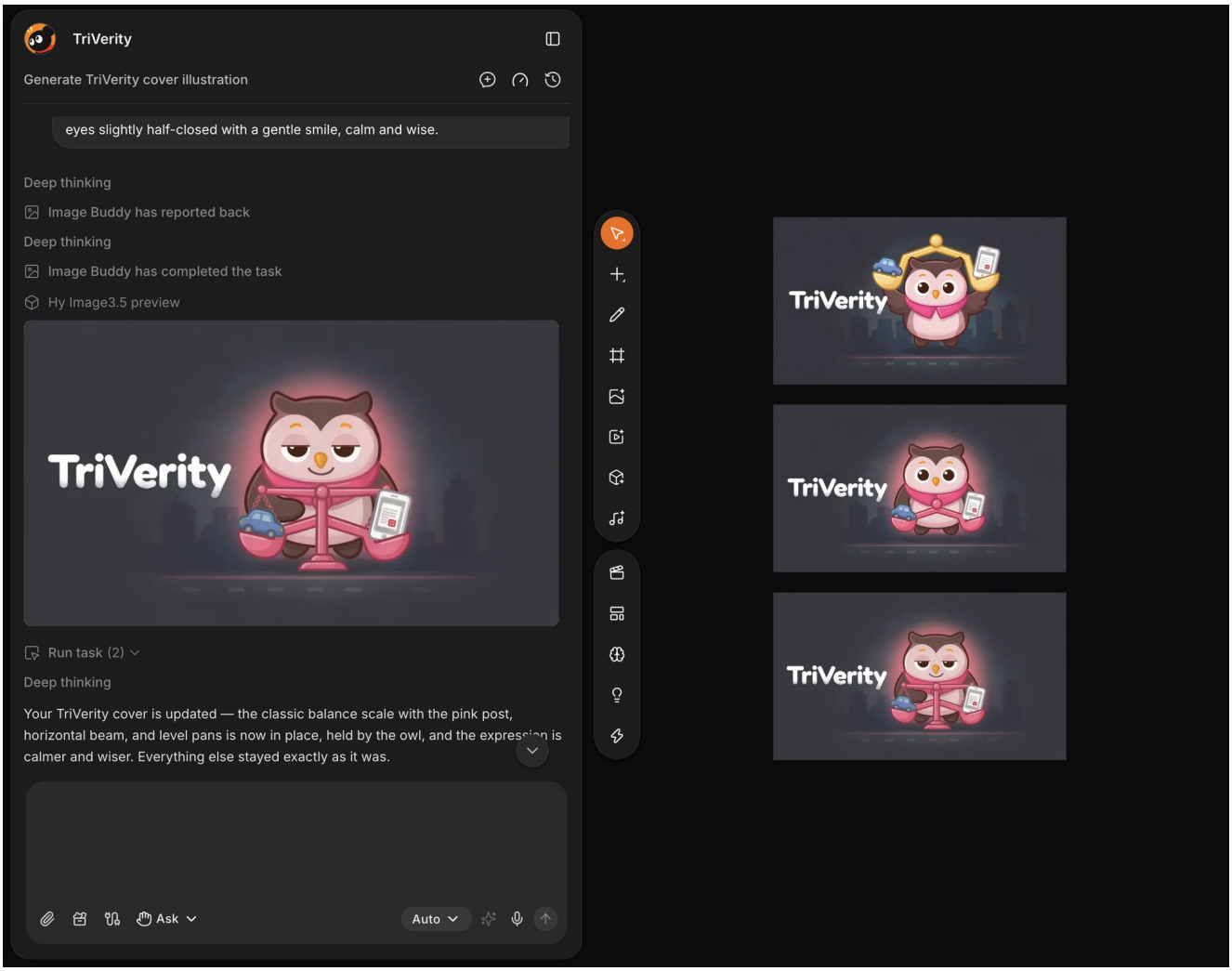


Figure 13. Second revision. A classic balance scale with level pans and a calmer, wiser expression; Miora produced several variants.

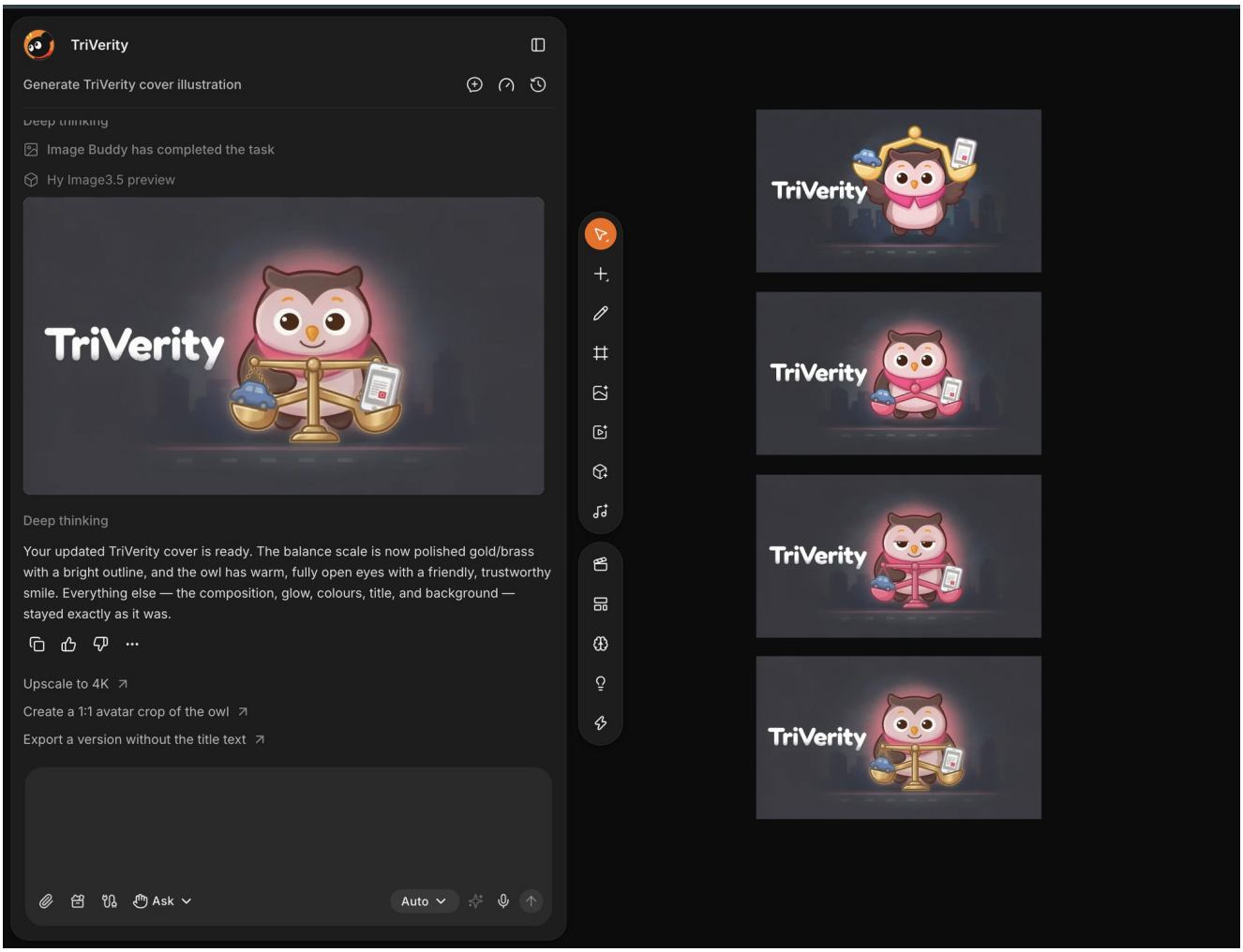


Figure 14. Final revision. A polished brass scale and an open-eyed owl; this version became our cover.



Figure 15. Final cover image (380 × 216). The cover submitted with the project.

Appendix: the full CodeBuddy session

All 12 prompts from the exported CodeBuddy conversation “Add shared LLM client with offline fallback” (Craft mode, 25–26 Sep 2026, Singapore time), summarised, with the number of tool calls CodeBuddy made for each (file reads and edits, terminal commands): 157 in total.

#	When (SGT)	Tool calls	What we asked CodeBuddy to do
1	25 Sep 20:05	16	Add the shared LLM client complete(system, user), with an offline placeholder and mocked tests
2	25 Sep 20:08	12	Rewrite the rider and driver advocates to use the LLM client and cite policy rule IDs
3	25 Sep 20:13	23	Review and fix the advocates: HTTP-error fallback, JSON mode, code-fence stripping, valid rule IDs
4	25 Sep 20:15	3	Push the code to the branch
5	25 Sep 20:19	18	Make the advocates robust to any malformed LLM output; record whether the AI or a fallback produced each case
6	25 Sep 20:46	2	Install the Tencent TokenHub CLI from its install guide
7	25 Sep 20:48	0	Which Hunyuan (Hy) model is best for this application?
8	25 Sep 20:48	1	Choose the suggested option
9	26 Sep 18:13	29	Add the deterministic Fraud & Bad-Faith Detection agent (assess_risk, RiskAssessment)
10	26 Sep 18:18	21	Show the fraud assessment in the UI: workflow step, risk panel, coloured badges; update docs
11	26 Sep 18:22	5	Commit and push
12	26 Sep 18:24	27	Fix the fraud branch: double-wrapped escalations, keep fraud signals out of party messages